

Souvenirs

Amaru đang mua đồ lưu niệm ở một cửa hàng nước ngoài. Có N **loại** đồ lưu niệm. Có vô số đồ lưu niệm của từng loại được bày bán trong cửa hàng.

Mỗi loại đồ lưu niệm đều có giá cố định. Cụ thể, một món đồ lưu niệm loại i ($0 \leq i < N$) có giá là $P[i]$ đồng tiền xu, trong đó $P[i]$ là số nguyên dương.

Amaru biết rằng các loại đồ lưu niệm được sắp xếp theo thứ tự giảm dần theo giá, và giá của các đồ lưu niệm cũng khác nhau. Cụ thể, $P[0] > P[1] > \dots > P[N-1] > 0$. Hơn nữa, anh ta có thể biết được giá trị của $P[0]$. Thật không may, Amaru không có thông tin nào khác về giá của các món quà lưu niệm.

Để mua một số đồ lưu niệm, Amaru sẽ thực hiện một số giao dịch với người bán.

Mỗi giao dịch bao gồm các bước sau:

1. Amaru đưa một số lượng (dương) đồng tiền xu cho người bán.
2. Người bán đặt những đồng xu này thành một đồng trên bàn ở phòng phía sau, nơi Amaru không thể nhìn thấy chúng.
3. Người bán xem xét lần lượt từng loại đồ lưu niệm $0, 1, \dots, N-1$ theo thứ tự đó. Mỗi loại được xem xét **đúng một lần** với mỗi giao dịch.
 - Khi xem xét loại quà lưu niệm i , nếu số lượng đồng xu hiện tại trong đồng tiền ít nhất là $P[i]$, thì
 - người bán lấy đi $P[i]$ đồng xu khỏi đồng tiền, và
 - người bán đặt một món đồ lưu niệm loại i lên bàn.
4. Người bán đưa cho Amaru toàn bộ số đồng xu còn lại trong đồng và tất cả đồ lưu niệm trên bàn.

Lưu ý rằng không có đồng tiền xu hoặc đồ lưu niệm nào trên bàn trước khi bắt đầu một giao dịch.

Nhiệm vụ của bạn là hướng dẫn Amaru thực hiện một số giao dịch để:

- trong mỗi giao dịch, anh ấy mua **ít nhất một** đồ lưu niệm và
- cuối cùng, anh ấy mua **chính xác** i đồ lưu niệm loại i , với mỗi i sao cho $0 \leq i < N$. Lưu ý rằng điều này có nghĩa là Amaru không cần mua bất kì món quà lưu niệm loại 0 nào.

Amaru không cần phải tối thiểu hoá số lượng giao dịch và có không giới hạn số lượng đồng xu.

Chi tiết cài đặt

Bạn cần cài đặt hàm sau.

```
void buy_souvenirs(int N, long long P0)
```

- N : số lượng loại quà lưu niệm.
- $P0$: giá trị của $P[0]$.
- Hàm này được gọi đúng một lần cho mỗi trường hợp thử nghiệm.

Hàm trên có thể thực hiện các lời gọi đến hàm sau để hướng dẫn Amaru thực hiện một giao dịch:

```
std::pair<std::vector<int>, long long> transaction(long long M)
```

- M : số đồng xu mà Amaru đưa cho người bán.
- Hàm trả về một cặp. Phần tử đầu tiên của cặp là một mảng L , chứa các loại quà lưu niệm đã được mua (theo thứ tự tăng dần). Phần tử thứ hai là một số nguyên R , số lượng đồng xu được trả lại cho Amaru sau lần giao dịch.
- Yêu cầu $P[0] > M \geq P[N - 1]$. Điều kiện $P[0] > M$ đảm bảo rằng Amaru không mua bất kì món quà lưu niệm nào thuộc loại 0 và $M \geq P[N - 1]$ đảm bảo rằng Amaru mua ít nhất một món quà lưu niệm. Nếu những điều kiện này không thoả mãn, lời giải của bạn sẽ nhận được phản hồi `Output isn't correct: Invalid argument`. Lưu ý rằng trái ngược với $P[0]$, giá trị của $P[N - 1]$ không được cung cấp trong dữ liệu đầu vào.
- Hàm này có thể được gọi tối đa 5000 lần trong mỗi trường hợp thử nghiệm.

Cách thức hoạt động của trình chấm là **không thích ứng**. Điều này có nghĩa là chuỗi giá P được cố định trước khi hàm `buy_souvenirs` được gọi.

Các ràng buộc

- $2 \leq N \leq 100$
- $1 \leq P[i] \leq 10^{15}$ với mỗi i sao cho $0 \leq i < N$.
- $P[i] > P[i + 1]$ với mỗi i sao cho $0 \leq i < N - 1$.

Các subtask

Subtask	Điểm	Các ràng buộc bổ sung
1	4	$N = 2$
2	3	$P[i] = N - i$ với mỗi i sao cho $0 \leq i < N$.
3	14	$P[i] \leq P[i + 1] + 2$ với mỗi i sao cho $0 \leq i < N - 1$.
4	18	$N = 3$
5	28	$P[i + 1] + P[i + 2] \leq P[i]$ với mỗi i sao cho $0 \leq i < N - 2$. $P[i] \leq 2 \cdot P[i + 1]$ với mỗi i sao cho $0 \leq i < N - 1$.
6	33	Không có ràng buộc nào thêm.

Ví dụ

Xét lời gọi hàm sau.

```
buy_souvenirs(3, 4)
```

Có $N = 3$ loại đồ lưu niệm và $P[0] = 4$. Lưu ý rằng chỉ có ba chuỗi giá P có thể xảy ra: $[4, 3, 2]$, $[4, 3, 1]$ và $[4, 2, 1]$.

Giả sử hàm `buy_souvenirs` gọi `transaction(2)`. Giả sử lời gọi trả về $([2], 1)$, nghĩa là Amaru đã mua một món quà lưu niệm loại 2 và người bán trả lại cho anh ta 1 đồng xu. Lưu ý rằng điều này cho phép chúng ta suy ra $P = [4, 3, 1]$, vì:

- Với $P = [4, 3, 2]$, `transaction(2)` sẽ trả về $([2], 0)$.
- Với $P = [4, 2, 1]$, `transaction(2)` sẽ trả về $([1], 0)$.

Sau đó hàm `buy_souvenirs` có thể gọi `transaction(3)`, trả về $([1], 0)$, nghĩa là Amaru đã mua một món quà lưu niệm loại 1 và người bán trả lại cho anh ta 0 đồng xu. Đến khi đó, tổng cộng anh ấy đã mua một món quà lưu niệm loại 1 và một món quà lưu niệm loại 2.

Cuối cùng, hàm `buy_souvenirs` có thể gọi `transaction(1)`, trả về $([2], 0)$, nghĩa là Amaru đã mua một món quà lưu niệm loại 2. Lưu ý rằng chúng ta cũng có thể sử dụng `transaction(2)` ở đây. Tại thời điểm này, tổng cộng Amaru có một món quà lưu niệm loại 1 và hai món quà lưu niệm loại 2 theo yêu cầu.

Trình chấm mẫu

Định dạng dữ liệu vào:

N

$P[0] \ P[1] \ \dots \ P[N-1]$

Định dạng kết quả ra:

$Q[0] \ Q[1] \ \dots \ Q[N-1]$

Với $Q[i]$ là tổng số đồ lưu niệm loại i đã mua với mỗi i sao cho $0 \leq i < N$.